

Week 6 Assignment

Spark Architecture, Lazy Evaluation, Data Processing and File Format Optimization using Apache Spark

Submitted By: Aditya Kumar

Internship: Celebal Technologies – Data Engineering

Objective

The objective of this assignment is to understand the architecture of Apache Spark and its execution model while performing efficient data processing using Spark Data Frames. The assignment covers Spark Architecture, Lazy Evaluation, DAG (Lineage Graph), schema handling, filtering, transformations, actions, file format optimization (CSV and Parquet), null handling, and building an end-to-end data processing pipeline.

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Answer -

Driver -

The Driver is the main process of a Spark application. It is responsible for creating the SparkSession, converting user code into execution plans (DAG), scheduling tasks, and coordinating the overall execution. It also collects the final results from the executors.

Cluster Manager -

The Cluster Manager is responsible for managing the resources of the cluster. It allocates CPU cores and memory to Spark applications and launches executors on worker nodes. Spark can work with different cluster managers such as Standalone, YARN, Mesos, and Kubernetes.

Executor -

Executors are worker processes that run on the worker nodes. They execute the tasks assigned by the Driver, perform computations, store intermediate data in memory or disk, and return the results back to the Driver.

Working Flow

1. The Driver receives the Spark application.
 2. The Cluster Manager allocates the required resources.
 3. Executors are launched on worker nodes.
 4. Executors process the assigned tasks.
 5. Results are sent back to the Driver.
-

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Answer -

Lazy Evaluation -

Lazy Evaluation is a core optimization feature of Apache Spark. When transformations such as `filter()`, `select()`, or `withColumn()` are applied, Spark does not execute them immediately. Instead, it records these operations in a **Directed Acyclic Graph (DAG)**, also known as the **Lineage Graph**. Execution begins only when an **Action** such as `show()`, `count()`, `collect()`, or `write()` is called.

How It Improves Performance

- Spark combines multiple transformations into a single optimized execution plan.
- It avoids unnecessary intermediate computations.
- It minimizes disk I/O by processing data efficiently in memory.
- It optimizes task execution using the DAG Scheduler.
- It executes only the operations required to produce the final result.

Example

```
df.filter(df["price"] > 100) \
  .select("product_id", "price") \
  .show()
```

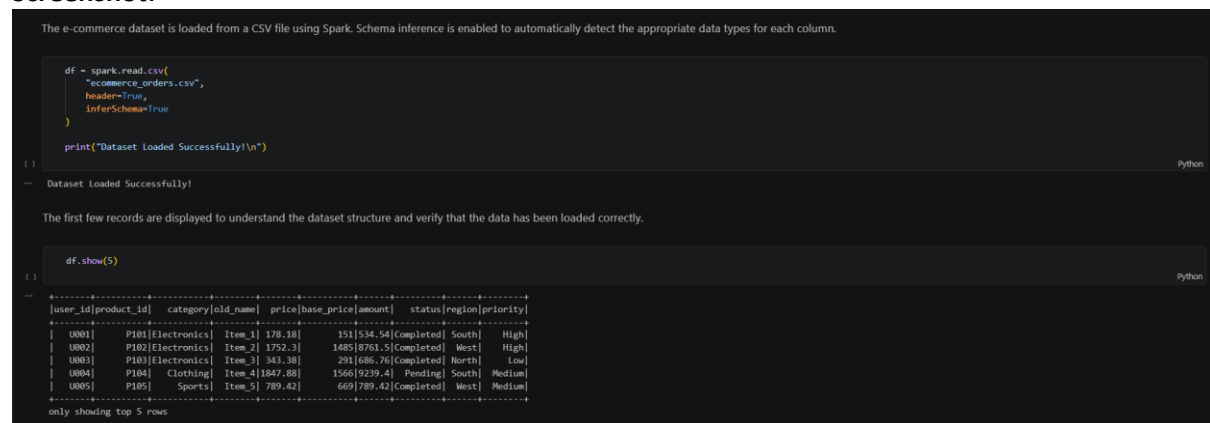
In this example, the `filter()` and `select()` transformations are **not executed immediately**. Spark records them in the DAG and performs both operations together only when `show()` is called.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

Answer -

The following Spark command reads a CSV file while treating the first row as the header and automatically inferring the data types of each column using `inferSchema=True`.

Screenshot:



The screenshot shows a Jupyter Notebook interface with two code cells. The first cell contains the code to read a CSV file with schema inference enabled. The second cell displays the first five rows of the loaded dataset.

```
df = spark.read.csv(
    "ecommerce_orders.csv",
    header=True,
    inferSchema=True
)

print("Dataset Loaded Successfully!\n")
```

Dataset Loaded Successfully!

The first few records are displayed to understand the dataset structure and verify that the data has been loaded correctly.

```
df.show(5)
```

user_id	product_id	category	old_name	price	base_price	amount	status	region	priority
U001	P101	Electronics	Item_1	178.18	151	534.54	Completed	South	High
U002	P102	Electronics	Item_2	1752.3	1485	8761.5	Completed	West	High
U003	P103	Electronics	Item_3	343.38	291	1686.76	Completed	North	Low
U004	P104	Clothing	Item_4	1847.88	1566	9239.4	Pending	South	Medium
U005	P105	Sports	Item_5	789.42	669	789.42	Completed	West	Medium

only showing top 5 rows

Explanation

- `spark.read.csv()` reads the CSV file.
 - `header=True` treats the first row as column names.
 - `inferSchema=True` automatically detects the appropriate data type for each column.
 - `show(5)` displays the first five records of the DataFrame.
-

Q4: Why is Parquet generally preferred over CSV for analytical workloads in Spark?

Answer -

Parquet is a **columnar storage format**, whereas CSV is a plain text row-based format. For analytical workloads, Spark prefers Parquet because it provides better performance, efficient storage, and optimized query execution.

Advantages of Parquet over CSV

- **Columnar Storage:** Only the required columns are read during query execution, reducing I/O operations.
- **Compression:** Parquet supports efficient compression techniques, reducing storage space.
- **Schema Preservation:** Data types are stored within the file, eliminating the need for schema inference every time the file is read.
- **Predicate Pushdown:** Spark reads only the necessary data blocks, improving query performance.
- **Faster Query Execution:** Reading fewer columns and compressed data significantly improves analytical performance.

Comparison

CSV	Parquet
Row-based storage	Column-based storage
Larger file size	Smaller compressed files
No schema information	Schema stored with data
Slower analytical queries	Faster analytical queries
Reads entire rows	Reads only required columns

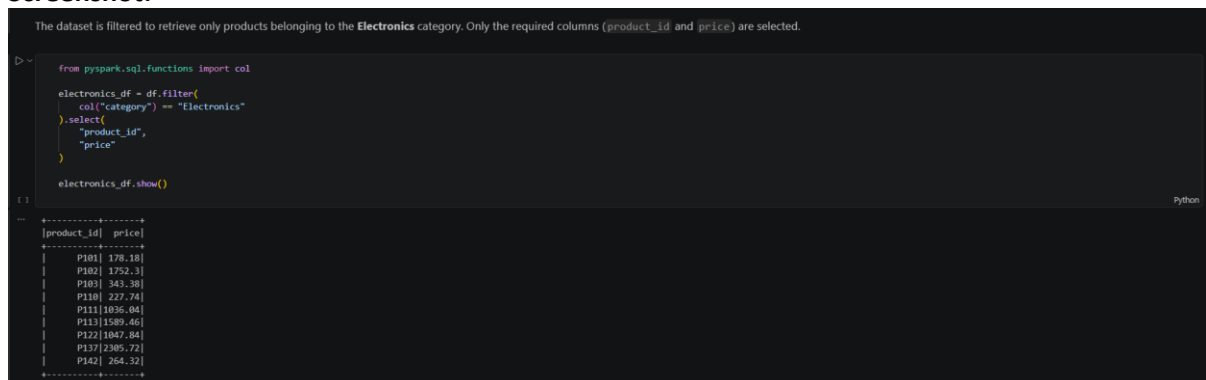
Therefore, Parquet is the preferred storage format for large-scale analytical workloads in Apache Spark.

Q5: Load a CSV dataset into a Spark DataFrame and display only the product_id and price columns where the category is Electronics.

Answer -

The following Spark code filters the dataset to retrieve only the products belonging to the **Electronics** category and displays only the product_id and price columns.

Screenshot:



```
The dataset is filtered to retrieve only products belonging to the Electronics category. Only the required columns (product_id and price) are selected.
```

```
from pyspark.sql.functions import col

electronics_df = df.filter(
    col("category") == "Electronics"
).select(
    "product_id",
    "price"
)

electronics_df.show()
```

```
--
[product_id| price]
-----
| P101| 178.18|
| P102| 1752.3|
| P103| 343.38|
| P110| 227.74|
| P111| 1036.04|
| P113| 1589.46|
| P122| 1047.84|
| P137| 2385.72|
| P142| 264.32|
-----
```

Explanation

- filter() is used to retrieve only rows where the category is **Electronics**.
 - select() displays only the required columns.
 - show() displays the filtered DataFrame.
-

Q6: Rename the column old_name to new_name and cast the price column to DoubleType.

Answer -

Spark provides withColumnRenamed() to rename existing columns and cast() to convert data types. In this example, the column old_name is renamed to new_name, and the price column is cast to DoubleType.

Screenshot:

```
This step demonstrates DataFrame modification by:
• Renaming a column
• Casting the data type of an existing column
• Verifying the updated schema

from pyspark.sql.types import StringType, DoubleType

df_string = df.withColumn(
    "price",
    col("price").cast(StringType())
)

updated_df = df_string.withColumnRenamed(
    "old_name",
    "new_name"
).withColumn(
    "price",
    col("price").cast(DoubleType())
)

updated_df.printSchema()
updated_df.show(5)
```

```
-- root
|-- user_id: string (nullable = true)
|-- product_id: string (nullable = true)
|-- category: string (nullable = true)
|-- new_name: string (nullable = true)
|-- price: double (nullable = true)
|-- base_price: integer (nullable = true)
|-- amount: double (nullable = true)
|-- status: string (nullable = true)
|-- region: string (nullable = true)
|-- priority: string (nullable = true)

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|user_id|product_id|category|new_name|price|base_price|amount|status|region|priority|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 0001 | P101 | Electronics | Item_1 | 178.18 | 151 | 534.54 | Completed | South | High |
| 0002 | P102 | Electronics | Item_2 | 1752.3 | 1485 | 8761.5 | Completed | West | High |
| 0003 | P103 | Electronics | Item_3 | 343.38 | 291 | 686.76 | Completed | North | Low |
| 0004 | P104 | Clothing | Item_4 | 1847.88 | 1566 | 9239.4 | Pending | South | Medium |
| 0005 | P105 | Sports | Item_5 | 789.42 | 669 | 789.42 | Completed | West | Medium |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

Explanation

- withColumnRenamed() changes the column name from old_name to new_name.
- cast(DoubleType()) converts the price column to the DoubleType data type.
- printSchema() verifies the updated schema.
- show(5) displays the modified DataFrame.

Q7: What is the Lineage Graph in Spark, and how does it help in fault tolerance?

Answer -

Lineage Graph

The Lineage Graph, also known as the **Directed Acyclic Graph (DAG)**, is a record of all the transformations applied to a Spark DataFrame or RDD. Instead of storing multiple copies of intermediate data, Spark records the sequence of operations performed on the data.

Role in Fault Tolerance

Spark uses the Lineage Graph to recover lost data automatically. If a partition is lost due to executor failure, Spark does not reload the entire dataset. Instead, it recomputes only the missing partition by re-executing the transformations recorded in the Lineage Graph.

Advantages

- Enables automatic fault recovery.
- Eliminates the need for data replication.
- Recomputes only the lost partitions.
- Improves reliability while reducing storage overhead.

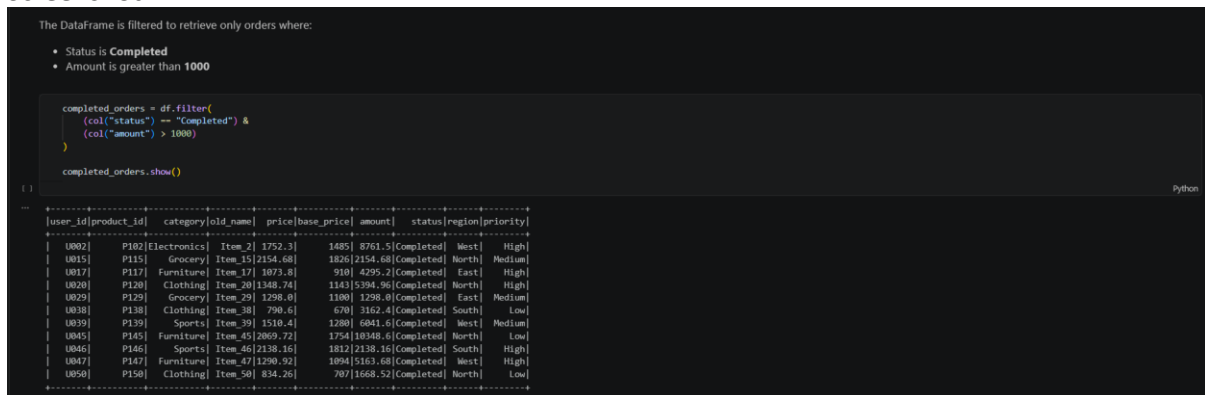
Therefore, the Lineage Graph plays a crucial role in Spark's fault-tolerant architecture.

Q8: Filter all orders where status = "Completed" and amount > 1000.

Answer -

The following Spark code filters the dataset to retrieve only the orders where the **status** is **Completed** and the **amount** is greater than **1000**.

Screenshot:



```
The DataFrame is filtered to retrieve only orders where:
```

- Status is Completed
- Amount is greater than 1000

```
completed_orders = df.filter(
    (col("status") == "Completed") &
    (col("amount") > 1000)
)

completed_orders.show()
```

user_id	product_id	category	old_name	price	base_price	amount	status	region	priority
U002	P102	Electronics	Item_2	1752.3	1485	8761.5	Completed	West	High
U015	P115	Grocery	Item_15	2154.68	1826	2154.68	Completed	North	Medium
U017	P117	Furniture	Item_17	1073.8	910	4295.2	Completed	East	High
U020	P120	Clothing	Item_20	1348.74	1143	5394.96	Completed	North	High
U029	P129	Grocery	Item_29	1298.0	1100	1298.0	Completed	East	Medium
U038	P138	Clothing	Item_38	790.6	670	3162.4	Completed	South	Low
U039	P139	Sports	Item_39	1510.4	1280	6841.6	Completed	West	Medium
U045	P145	Furniture	Item_45	2069.72	1754	10348.6	Completed	North	Low
U046	P146	Sports	Item_46	2138.16	1812	2138.16	Completed	South	High
U047	P147	Furniture	Item_47	1290.92	1094	5163.68	Completed	West	High
U050	P150	Clothing	Item_50	834.25	707	1668.52	Completed	North	Low

Explanation

- filter() is used to apply multiple filtering conditions.
- The logical **AND (&)** operator combines both conditions.
- Only records satisfying both conditions are returned.
- show() displays the filtered DataFrame.

Q9: What is Predicate Pushdown in Spark, and why does it improve query performance?

Answer -

Predicate Pushdown

Predicate Pushdown is an optimization technique used by Spark while reading data from storage formats such as **Parquet** and **ORC**. Instead of loading the entire dataset into memory, Spark pushes the filtering conditions down to the storage layer.

Benefits

- Reads only the required data blocks.
- Reduces disk I/O operations.
- Improves query execution time.
- Decreases memory usage.
- Enhances overall Spark performance.

Example

If the query requests only records where:

category = "Electronics"

Spark reads only the relevant data blocks instead of scanning the complete dataset.

This optimization is one of the primary reasons why columnar formats such as **Parquet** perform significantly better than CSV for analytical workloads.

Q10: Add a new column called `final_price` by applying an 18% tax on the `base_price` column.

Answer -

The following Spark code creates a new column named `final_price` by applying an **18% tax** to the values in the `base_price` column.

Screenshot:

```
A new column named final_price is created by applying an 18% tax to the base_price column.
```

```
> final_price_df = df.withColumn(
  "final_price",
  col("base_price") * 1.18
)

final_price_df.show(5)
```

user_id	product_id	category	old_name	price	base_price	amount	status	region	priority	final_price
U001	P101	Electronics	Item_1	178.18	151	534.54	Completed	South	High	178.17999999999998
U002	P102	Electronics	Item_2	1752.3	1485	18761.51	Completed	West	High	1752.3
U003	P103	Electronics	Item_3	343.38	291	686.76	Completed	North	Low	343.38
U004	P104	Clothing	Item_4	1847.88	1566	9239.4	Pending	South	Medium	1847.8799999999999
U005	P105	Sports	Item_5	789.42	669	789.42	Completed	West	Medium	789.42

only showing top 5 rows

Explanation

- `withColumn()` is used to create a new column in the DataFrame.
- The new column `final_price` is calculated by multiplying the `base_price` column by **1.18**.
- `show(5)` displays the updated DataFrame with the newly created column.

Q11: Differentiate between Transformations and Actions in Spark with suitable examples.

Answer -

Spark operations are broadly classified into **Transformations** and **Actions**.

Transformations

Transformations create a new DataFrame or RDD from an existing one without executing immediately. They are **lazy** in nature and are recorded in the **Lineage Graph (DAG)**.

Examples: `select()`, `filter()`, `withColumn()`, `drop()`, `groupBy()`

Actions

Actions trigger the execution of all previously defined transformations and produce the final output.

Examples: `show()`, `count()`, `collect()`, `first()`, `write()`

Example

```
transformation_df = df.select("user_id", "product_id")
```

```
transformation_df.show()
```

In the above example:

- `select()` is a **Transformation**.
- `show()` is an **Action**.

Screenshot:

```
Transformations create a new DataFrame without immediate execution.
Actions trigger the execution of the entire Spark job and produce a result.
```

```
> print("Transformation Example:")
transformation_df = df.select("user_id", "product_id")

print("Action Example:")
transformation_df.show(5)
```

```
Transformation Example:
Action Example:
+-----+
|user_id|product_id|
+-----+
| U001  | P101     |
| U002  | P102     |
| U003  | P103     |
| U004  | P104     |
| U005  | P105     |
+-----+
```

only showing top 5 rows

Q12: Build a Spark pipeline that reads a CSV file, converts it to Parquet, reads the Parquet file, filters out rows where user_id is null, and writes the result into a new CSV file.

Answer -

The following Spark pipeline demonstrates a complete data processing workflow.

Pipeline Steps

1. Read the CSV file.
2. Convert the DataFrame to Parquet format.
3. Read the generated Parquet file.
4. Filter out rows where user_id is null.
5. Write the processed DataFrame to a new CSV file.
6. Verify the generated output files.

Screenshot:

```
The Parquet file generated in the previous step is read back into a Spark DataFrame to verify successful storage.

parquet_df = spark.read.parquet("input_parquet")
parquet_df.show(5)
```

```
[ ] Python
```

user_id	product_id	category	old_name	price	base_price	amount	status	region	priority
U001	P101	Electronics	Item_1	178.18	151	534.54	Completed	South	High
U002	P102	Electronics	Item_2	1752.3	1485	8761.5	Completed	West	High
U003	P103	Electronics	Item_3	343.38	291	686.76	Completed	North	Low
U004	P104	Clothing	Item_4	1847.88	1566	9239.4	Pending	South	Medium
U005	P105	Sports	Item_5	789.42	669	789.42	Completed	West	Medium

only showing top 5 rows

```
The generated Parquet and CSV output folders are inspected to verify successful execution of the Spark pipeline.

import os

print("Files inside input_parquet:")
print(os.listdir("input_parquet"))

print("\nFiles inside output_csv:")
print(os.listdir("output_csv"))
```

```
[ ] Python
```

```
Files inside input_parquet:
['_SUCCESS.crc', '.part-00000-15f4e03d-9d2f-493c-b5c8-625ebf5e5459-c000.snappy.parquet.crc', 'part-00000-15f4e03d-9d2f-493c-b5c8-625ebf5e5459-c000.snappy.parquet', '_SUCCESS']

Files inside output_csv:
['_SUCCESS.crc', 'part-00000-b3166943-8c39-4951-baad-21daa285a545-c000.csv', '_SUCCESS', '.part-00000-b3166943-8c39-4951-baad-21daa285a545-c000.csv.crc']
```

```
The generated CSV file is loaded back into Spark to verify that the pipeline has successfully written the processed data.

output_df = spark.read.csv(
    "output_csv",
    header=True,
    inferSchema=True
)

output_df.show(5)
```

```
[ ] Python
```

user_id	product_id	category	old_name	price	base_price	amount	status	region	priority
U001	P101	Electronics	Item_1	178.18	151	534.54	Completed	South	High
U002	P102	Electronics	Item_2	1752.3	1485	8761.5	Completed	West	High
U003	P103	Electronics	Item_3	343.38	291	686.76	Completed	North	Low
U004	P104	Clothing	Item_4	1847.88	1566	9239.4	Pending	South	Medium
U005	P105	Sports	Item_5	789.42	669	789.42	Completed	West	Medium

only showing top 5 rows

Explanation

- The original CSV dataset is converted into **Parquet** format.
- The Parquet file is read back into Spark.
- Records containing null values in the **user_id** column are removed.
- The cleaned DataFrame is written into a new CSV file.
- The generated Parquet and CSV folders are verified to ensure successful execution of the Spark pipeline.

Q13: Differentiate between Client Mode and Cluster Mode in Spark Architecture.

Answer -

Client Mode

In **Client Mode**, the Driver program runs on the client machine from which the Spark application is submitted. The Executors run on the worker nodes of the cluster. Since the Driver is outside the cluster, the client machine must remain active throughout the execution of the application.

Cluster Mode

In **Cluster Mode**, the Driver program runs inside the cluster on one of the worker nodes. The Cluster Manager is responsible for launching both the Driver and the Executors. The client can disconnect after submitting the application because the cluster manages the execution independently.

Difference Between Client Mode and Cluster Mode

Client Mode	Cluster Mode
Driver runs on the client machine	Driver runs inside the cluster
Client machine must remain active	Client can disconnect after job submission
Suitable for development and testing	Suitable for production environments
Dependent on client availability	Managed entirely by the cluster

Q14: Write a query to filter a dataset for rows where the region is "North" OR the priority is "High".

Answer -

The following query filters the DataFrame to retrieve all records where the **region** is **North** or the **priority** is **High**.

Screenshot:

```
The DataFrame is filtered to retrieve records where:
• Region is North
• OR Priority is High

north_or_high = df.filter(
    (col("region") == "North") |
    (col("priority") == "High")
)

north_or_high.show()
```

user_id	product_id	category	old_name	price	base_price	amount	status	region	priority
U001	P101	Electronics	Item_1	178.18	151	534.54	Completed	South	High
U002	P102	Electronics	Item_2	1752.3	1485	8761.5	Completed	West	High
U003	P103	Electronics	Item_3	343.38	291	686.76	Completed	North	Low
U006	P106	Grocery	Item_6	493.24	418	986.48	Pending	North	High
U007	P107	Furniture	Item_7	251.64	288	1854.92	Pending	East	High
U008	P108	Furniture	Item_8	1413.64	1198	1413.64	Pending	North	Low
NULL	P110	Electronics	Item_10	227.74	193	455.48	Pending	North	High
U011	P111	Electronics	Item_11	1036.04	878	3108.12	Pending	East	High
U013	P113	Electronics	Item_13	1589.46	1347	3178.92	Cancelled	South	High
U015	P115	Grocery	Item_15	2154.68	1826	2154.68	Completed	North	Medium
U016	P116	Furniture	Item_16	764.64	648	764.64	Completed	East	High
U017	P117	Furniture	Item_17	1873.8	910	4295.2	Completed	East	High
U019	P119	Sports	Item_19	1152.86	977	5764.3	Pending	East	High
U020	P120	Clothing	Item_20	1348.74	1143	5394.96	Completed	North	High
U023	P123	Grocery	Item_23	1454.94	1223	1454.94	Cancelled	North	Low
U024	P124	Sports	Item_24	1931.66	1637	5794.98	Cancelled	East	High
NULL	P125	Grocery	Item_25	1168.2	990	2336.4	Pending	North	Low
U026	P126	Grocery	Item_26	1327.5	1125	2655.8	Cancelled	North	Low
U027	P127	Grocery	Item_27	2151.14	1823	18755.7	Cancelled	South	High
U028	P128	Grocery	Item_28	1959.98	1661	3919.96	Cancelled	North	Low

only showing top 20 rows

Explanation

- filter() is used to apply conditions to the DataFrame.
- col("region") == "North" selects records belonging to the North region.
- col("priority") == "High" selects records with High priority.
- The | (OR) operator ensures that records satisfying either condition are returned.
- show() displays the filtered results.

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Answer -

`.show(5)`

The `.show(5)` function displays only the first five rows of a DataFrame. It retrieves a small subset of data for inspection, making it memory-efficient and suitable for exploring large datasets.

`.collect()`

The `.collect()` function retrieves **all** the records from the executors and transfers them to the Driver program as a local collection. For very large datasets, this can consume a significant amount of memory and may cause the Driver to crash with an **Out of Memory (OOM)** error.

Why `.show(5)` is Safer

- Retrieves only a few rows instead of the entire dataset.
- Consumes very little memory on the Driver.
- Prevents Out of Memory (OOM) errors.
- Faster for inspecting data.
- Recommended for exploring large or multi-terabyte datasets.

Difference Between `.show(5)` and `.collect()`

<code>.show(5)</code>	<code>.collect()</code>
Displays only the first five rows	Retrieves all rows
Memory efficient	Memory intensive
Suitable for large datasets	Suitable only for small datasets

Screenshot:

```
When exploring large datasets, show() is preferred because it retrieves only a small number of records.
Using collect() on very large datasets can cause excessive memory usage because it transfers all data to the Driver.

print("Using show():")
df.show(5)

print("\nUsing collect() on only 5 rows:")
small_data = df.limit(5).collect()

for row in small_data:
    print(row)

()

Using show():
+-----+-----+-----+-----+-----+-----+-----+-----+
|user_id|product_id|category|old_name|price|base_price|amount|status|region|priority|
+-----+-----+-----+-----+-----+-----+-----+-----+
| 0001 | P101 | Electronics | Item_1 | 178.18 | 151 | 534.54 | Completed | South | High |
| 0002 | P102 | Electronics | Item_2 | 1752.3 | 1485 | 8761.5 | Completed | West | High |
| 0003 | P103 | Electronics | Item_3 | 343.38 | 291 | 686.76 | Completed | North | Low |
| 0004 | P104 | Clothing | Item_4 | 1847.88 | 1566 | 9239.4 | Pending | South | Medium |
| 0005 | P105 | Sports | Item_5 | 789.42 | 669 | 789.42 | Completed | West | Medium |
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

Using collect() on only 5 rows:
Row(user_id='0001', product_id='P101', category='Electronics', old_name='Item_1', price=178.18, base_price=151, amount=534.54, status='Completed', region='South', priority='High')
Row(user_id='0002', product_id='P102', category='Electronics', old_name='Item_2', price=1752.3, base_price=1485, amount=8761.5, status='Completed', region='West', priority='High')
Row(user_id='0003', product_id='P103', category='Electronics', old_name='Item_3', price=343.38, base_price=291, amount=686.76, status='Completed', region='North', priority='Low')
Row(user_id='0004', product_id='P104', category='Clothing', old_name='Item_4', price=1847.88, base_price=1566, amount=9239.4, status='Pending', region='South', priority='Medium')
Row(user_id='0005', product_id='P105', category='Sports', old_name='Item_5', price=789.42, base_price=669, amount=789.42, status='Completed', region='West', priority='Medium')
```

Conclusion

Apache Spark provides a highly efficient distributed computing framework for processing large-scale datasets. Through this assignment, Spark Architecture, Lazy Evaluation, DAG (Lineage Graph), DataFrame transformations, filtering, schema handling, null handling, and file format optimization using **CSV** and **Parquet** were successfully implemented.

The practical implementation demonstrated how Spark optimizes execution using lazy evaluation, minimizes disk I/O through columnar storage, and efficiently processes data using DataFrame APIs.

The assignment also highlighted Spark best practices such as using `show()` instead of `collect()` for exploring large datasets and utilizing Parquet for improved analytical performance.

Overall, this assignment provided hands-on experience in building an end-to-end Spark data processing pipeline involving reading data, applying transformations, filtering records, handling null values, and writing processed data into optimized storage formats.